

中断

轮询和中断

如果需要在发生某个事件之后，需要执行一些特定的逻辑，为了实现这个需求，可以使用轮询或者中断。

轮询是一种朴素的实现方式，通过一个用不停歇的循环，在循环内部不断的判断某个事件是否发生，如果发生就跳转到对应的处理逻辑上即可。轮询是能够完成这个功能的，但是轮询不够优秀，永不停息的循环意味着永不停息的功耗。

现代处理器都提供了中断功能，这是一项由硬件支持的功能，当某个事件发生后，可以触发中断，一旦级别够高的中断被触发，那么核心无论现在在做什么，都要优先处理这个中断

中断是由硬件产生了事件，并且改变了程序的执行流程

NVIC

之前讲过这个NVIC属于内核外设，这块功能是由ARM公司设计的，交付的时候会随着核心一起交付给厂商。

NVIC 全称 Nested Vectored Interrupt Controller - 嵌套向量中断控制器

它接受来自硬件产生的硬中断（比如来自外设的中断）与来自处理器的软中断（比如执行代码出现的异常触发的中断等）

中断可以实现抢占式调度，在执行进程1的时候，来了一个比进程1优先级高的中断2，那核心就要跳过去处理中断2，如果中断2还没处理完，又来了一个优先级更高的中断3，那核心就要跳过去先处理中断3，处理完中断3，再回头处理中断2，处理完中断2，才能继续处理进程1

中断向量表

!!! 以STM32 Flash启动为例!!!

中断发生之后，由中断服务负责处理，这个中断服务本质上也就是一个函数，那么这个函数的入口在哪里？核心怎么知道如何去寻找某个中断对应的中断服务地址呢？

答案是依靠中断向量表，以Cortex-M内核的芯片的中断向量表为例：

存储器地址	中断向量表	异常编号
0x0000 004c	中断3向量	19
0x0000 0048	中断2向量	18
0x0000 0044	中断1向量	17
0x0000 0040	中断0向量	16 # 从第16项开始由芯片厂商定义
0x0000 003c	SysTick向量	15 # 前15项是固定的
0x0000 0038	PendSV向量	14 # 由ARM给出
0x0000 0034	未使用	13
0x0000 0030	调试监控向量	12
0x0000 002c	SVC向量	11
0x0000 0028	未使用	10
0x0000 0024	未使用	9
0x0000 0020	未使用	8
0x0000 001c	未使用	7
0x0000 0018	使用错误向量	6
0x0000 0014	总线错误向量	5
0x0000 0010	MemManage向量	4
0x0000 000c	HardManage向量	3
0x0000 0008	NMI向量	2
0x0000 0004	复位向量	1
0x0000 0000	MSP初始向量	0

可以看到，介绍启动流程时着重介绍过的MSP初始向量和复位向量也是中断向量表中的一部分，其实复位重启本身就是一种中断，而且是优先级最高的一个中断

一般而言，中断向量表是由芯片厂商提供的，如果你用的STM32的话，那就要去ST的文档里找完整的中断向量表了

我让deepseek写了一个STM32F103的启动代码，从中截取了一部分中断向量表：

```

# .....

/* ===== 复位向量 ===== */
.global Reset_Handler
.type Reset_Handler, %function
Reset_Handler:
    # 实现 Reset_Handler 中断服务
    # 省略实现 .....

/*===== SVC中断服务 =====*/
.global SVC_Handler
.type SVC_Handler, %function
SVC_Handler:
    # 实现SVC中断服务
    mov r0, #0xA5
    bx lr

/* ===== 向量表 ===== */
.section .isr_vector, "a", %progbits
.global g_pfnVectors
.type g_pfnVectors, %object
g_pfnVectors:
    .word _estack                /* 0x00 初始MSP*/
    .word Reset_Handler         /* 0x04 复位向量*/
    # ..... 省略中间的中断向量
    .word SVC_Handler           /* 0x2C SVC向量*/
    # ..... 省略后面的中断向量

```

尽管我省略了大部分内容，还是能看出来，中断向量表是由一系列函数入口地址构成的

当某个中断触发的时，CPU就会根据中断向量表里记录的中断服务入口进行中断处理；比如当复位中断触发的时候，就执行复位中断对应的中断服务Reset_Handler；当SVC中断发生的时候，就执行SVC中断对应的服务SVC_Handler

VTOR-向量表偏移寄存器

VTOR记录了向量表的基地址，中断服务的入口函数地址 = $[VTOR] + N * 4$ ，其中N是中断号

例如：要找11号中断——SVC中断服务的地址，就 $[VTOR] + 11 * 4$

VTOR默认值是0（也就是0x0000 0000）

为什么会存在VTOR？难道中断向量表不是一直在0x0000 0000吗？

答案是：系统中可能存在多个异常向量表

上电复位时，VTOR的值是默认值（0），此时中断向量表的起始地址是0x0000 0000，但是由于STM32把0x0000 0000映射到 0x08000000上的原因，实际上生效的是0x0800 0000的中断向量表，在之前的学习中，我们使用链接脚本把中断向量表放到0x0800 0000的原因在这里；

但是实际使用中，芯片不可能一直处于这个状态，比如我们使用STM32跑一个RTOS，那么在上电复位之后，程序的控制权就得交给RTOS了，而RTOS有自己的中断向量表，这时候就不能再使用上电之后使用的那个异常向量表了；

VTOR提供了一种很方便的切换异常向量表的方式，不管系统里到底存在几个异常向量表，只要修改VTOR就能让NVIC对应的异常向量表

使用中断

从软件中断开始入手

```
# 例如  
svc #0xA5
```

执行这行代码后，就会自动专跳到SVC_Handler中

